



RECUPERACIÓN DE INFORMACIÓN Y RECOMENDACIONES EN LA WEB

Curso 2026 – Facultad de Ingeniería (Udelar)

Entrega 1: Buscador Mejorado de Trámites de gub.uy

Búsqueda Híbrida, Reordenamiento Semántico y Asistencia RAG sobre Datos Abiertos de AGESIC

Autores (Grupo 07):

Belén Olivera C.I. 5.598.984-3
Nicolás Pereira C.I. 5.139.786-4

Docente:

Libertad Tansini

Montevideo, Uruguay
4 de octubre de 2026

Índice

1. Introducción 2

2. Definición del Problema 2

3. Enfoque de la Solución 3

4. Diseño de Arquitectura y Componentes 4

1. Introducción

Hoy en día, el portal `gub.uy/tramites` centraliza la información de cientos de trámites públicos de Uruguay. Sin embargo, cuando un ciudadano intenta buscar cómo hacer un trámite, la efectividad de la respuesta depende totalmente de las palabras exactas que ingrese en el buscador.

El objetivo de este proyecto es mejorar la experiencia de búsqueda en el catálogo de trámites del Estado, utilizando el dataset público de AGESIC disponible en `catalogodatos.gub.uy`. Para lograrlo, planteamos una solución de Recuperación de Información que combina búsqueda tradicional por palabras clave con búsqueda semántica, filtros prácticos (como costo o si el trámite es en línea) y un chatbot conversacional basado en RAG para explicar los requisitos de forma sencilla.

2. Definición del Problema

El buscador actual de `gub.uy` funciona principalmente buscando coincidencias exactas de texto (*keyword matching*). Esto genera varios problemas cotidianos:

En primer lugar, está el problema de la diferencia de vocabulario (*vocabulary mismatch*). Las personas suelen buscar usando un lenguaje cotidiano (por ejemplo, *"renovar cedula"* o *"renovar el carnet de salud"*), mientras que los documentos del Estado usan títulos formales (como *"Documento Nacional de Identidad"* o *"Carné de Salud Laboral"*). Si el usuario no usa la palabra precisa (por ejemplo un error de tipeo olvidando una letra), el buscador no encuentra el resultado correcto.

En segundo lugar, las búsquedas por texto no entienden la intención ni el contexto de la frase. Al buscar por sinónimos o consultas más largas, los motores tradicionales suelen devolver trámites irrelevantes o directamente ningún resultado.

Por último, los resultados actuales no permiten filtrar fácilmente por cuestiones prácticas del día a día, como saber si el trámite es gratuito, si se puede hacer 100 % por internet o a qué organismo pertenece. Además, para entender los requisitos o costos de un trámite, el usuario tiene que entrar y leer varios apartados por separado.

3. Enfoque de la Solución

Para resolver estas limitaciones, diseñamos un sistema de búsqueda en dos etapas (*two-stage retrieval*) junto con herramientas de filtrado y asistencia conversacional. La elección de cada componente responde a una necesidad concreta:

1. **Búsqueda Léxica (BM25):** Mantenemos el algoritmo tradicional BM25 porque es fundamental para encontrar nombres exactos de leyes, números de decretos o siglas institucionales (por ejemplo, *"BPS"* o *"IMPO"*).

2. **Búsqueda Semántica con Embeddings (k-NN):** Para solucionar el problema de los sinónimos y el lenguaje coloquial, transformamos los textos y la consulta a un espacio vectorial utilizando el modelo multilingüe `intfloat/multilingual-e5-small`. Esto nos permite comparar la consulta y los trámites por su significado conceptual y no solo por sus palabras.

3. **Fusión RRF (Reciprocal Rank Fusion):** Como la búsqueda léxica y la búsqueda por embeddings devuelven puntajes en escalas muy distintas, utilizamos el algoritmo RRF para combinar ambos ordenamientos de forma justa según sus posiciones relativas.

4. **Reordenamiento (Reranking):** Tomamos los mejores resultados devueltos por la fase híbrida y los pasamos por un modelo *cross-encoder* (`mmarco-mMiniLMv2-L12-H384-v1`). Este modelo compara la consulta directamente contra el contenido de cada trámite para reordenarlos y asegurar que los más relevantes queden bien arriba en la lista.

5. **Filtros por Facetas:** Extraemos metadatos clave del archivo CSV para permitir al usuario filtrar los resultados según su necesidad: si el trámite es gratuito o pago, si se hace de forma presencial o por internet, y el organismo que lo emite.

6. **Asistente Conversacional (RAG):** Para no obligar al usuario a leer todo el documento, implemetaremos una vista en donde usamos un modelo de lenguaje local (`llama3.2` corriendo en Ollama) que toma los mejores trámites encontrados y redacta una respuesta clara y resumida con los pasos a seguir, incluyendo siempre los links oficiales a `gub.uy`.

4. Diseño de Arquitectura y Componentes

El sistema está pensado con una arquitectura modular y desacoplada, utilizando Docker para facilitar su ejecución y despliegue local.

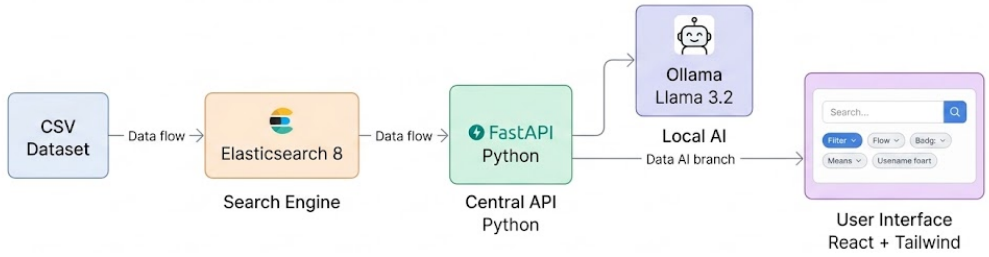


Figura 1: Esquema general de la arquitectura propuesta para el sistema.

Como se observa en la Figura 1, el flujo de trabajo se divide en los siguientes módulos:

- **Corpus de Datos (Catálogo de Trámites:** Es el archivo CSV descargado de la plataforma de datos abiertos de AGESIC, que contiene toda la información oficial estructurada y los textos de cada trámite.
- **Elasticsearch 8:** Es el motor encargado de guardar e indexar los datos. Configuramos un analizador en español (para pasar a minúsculas, quitar tildes, sacar palabras vacías y aplicar *stemming*) y guardamos los vectores para hacer búsquedas de similitud k-NN.
- **Backend en FastAPI:** Es una API en Python que recibe las consultas de la aplicación web, ejecuta la búsqueda híbrida, aplica la fusión RRF y el reranking, y manda el contexto al modelo de lenguaje. También incluye un índice BM25 local (`rank_bm25`) por si Elasticsearch no está disponible.
- **Servicio RAG con Ollama:** Ejecuta de forma local el modelo `llama3.2` para generar las respuestas del chatbot basándose únicamente en los trámites recuperados.
- **Interfaz Web (React + Tailwind CSS):** Es la aplicación con la que interactúa el usuario. Cuenta con la barra de búsqueda principal, panel de filtros por facetas, resaltado de palabras clave en los resultados y la ventana de chat.

Declaración sobre el Uso de Inteligencia Artificial

Aclaramos que para la elaboración de la presente entrega se utilizó asistencia de Modelos de Lenguaje de Gran Escala (LLMs). En particular, se empleó IA como herramienta de apoyo para el análisis y toma de decisiones sobre la arquitectura de solución más adecuada frente al problema planteado, así como para la generación del diagrama visual de arquitectura e iteraciones de revisión en la redacción y formato del informe. El diseño técnico, la selección del corpus y la definición conceptual del proyecto fueron propuestos y validados por los autores.

Referencias

[1] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

[2] AGESIC. (2024). *Catálogo de Trámites y Servicios del Estado*. Datos Abiertos de Uruguay. https://catalogodatos.gub.uy/dataset/agesic-guia-de-tramites

[3] Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal rank fusion outperforms rank aggregation methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval* (pp. 758–759).

[4] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (EMNLP).